

AI System Architecture

Designing Scalable, Reliable AI-Powered Systems

AWS/GCP Architecture Patterns | Anthropic Engineering Blog | 2024

Overview

Designing AI-powered systems requires extending classical system design with ML-specific concerns: data pipelines, model serving, context management, prompt engineering, and AI safety. This document covers the key patterns for building scalable, reliable AI systems.

1. AI System Design Framework

When designing an AI system, address these layers:

- User Interface:** How users interact: chat, API, embedded widget
- Orchestration Layer:** LLM routing, agent loops, context assembly, tool dispatch
- LLM Layer:** Model selection, prompt templates, caching, fallback models
- Memory & State:** Conversation history, vector store, key-value cache
- Tool/Integration Layer:** External APIs, databases, search engines, code execution
- Data Pipeline:** Ingestion, chunking, embedding, indexing
- Infrastructure:** Compute, storage, networking, monitoring, security

2. Design: Production RAG System

Architecture for a high-throughput enterprise RAG system:

Component	Technology Choice	Why
Document ingestion	Apache Kafka + Flink	Real-time pipeline, handles bursts
Chunking service	Custom Python + LangChain	Semantic + structural splitting
Embedding	voyage-3 or text-emb-3-large	High quality, supports long context
Vector store	Weaviate (distributed)	Filtered ANN, multi-tenancy, GraphQL
Sparse index	Elasticsearch BM25	Hybrid search for keyword precision
Reranker	Cohere Rerank v3	Precision boost before LLM context injection
LLM Gateway	LiteLLM proxy	Load balancing, cost tracking, fallbacks

Cache	Redis + GPTCache	Semantic cache; 30–70% cost savings
Monitoring	LangFuse + Prometheus	Trace LLM calls, track latency/cost

3. Agent System Design

For agentic AI systems that take multi-step actions, the key design decisions are:

- Orchestration model:** ReAct (Reason+Act), Plan-and-Execute, or DAG-based pipelines
- State management:** In-context window vs. external memory store (Redis, Postgres)
- Tool granularity:** Broad tools (run_code) vs. specific (read_file, write_file, list_dir)
- Safety guardrails:** Human-in-the-loop checkpoints, max_steps limits, sandboxed execution
- Error handling:** Retry with exponential backoff, fallback to simpler approach, human escalation
- Parallelism:** Fan-out parallel subagents for independent subtasks; fan-in for aggregation

4. Capacity Planning for LLM Systems

Key metrics for LLM system sizing:

Throughput: $\text{Tokens/sec} \approx \text{GPU_memory_bandwidth} / (2 * \text{bytes_per_param})$ per layer

- Context window limits: GPT-4 128k, Claude 3.5 Sonnet 200k, Gemini 1.5 Pro 1M tokens
- Latency targets: Interactive < 1s TTFT; Batch jobs < 60s total
- Cost modeling: input tokens cheaper than output (2–5x); cache reduces cost 90%+
- Concurrency: vLLM achieves 10–24x throughput over naive HuggingFace via PagedAttention
- Scaling pattern: Horizontal scaling behind load balancer; sticky routing for KV cache hits